



2024年度後期輪読会 第7回 [物体検出]

SSD

京都大学 工学部 情報学科

宮前明生

- SSD以前モデルの課題点
- SSDの推論
- SSDの学習
- 実験結果

- SSD以前モデルの課題点
- SSDの推論
- SSDの学習
- 実験結果

SSD以前のモデルの課題点

■ SSD以前の物体検出モデル（Faster R-CNNなど）

1. バウンディングボックスを検出する
2. 各ボックスに分類器を適用する

■ 課題点

- 計算量が多く、リアルタイム検出に向かない



■ SSDの概要

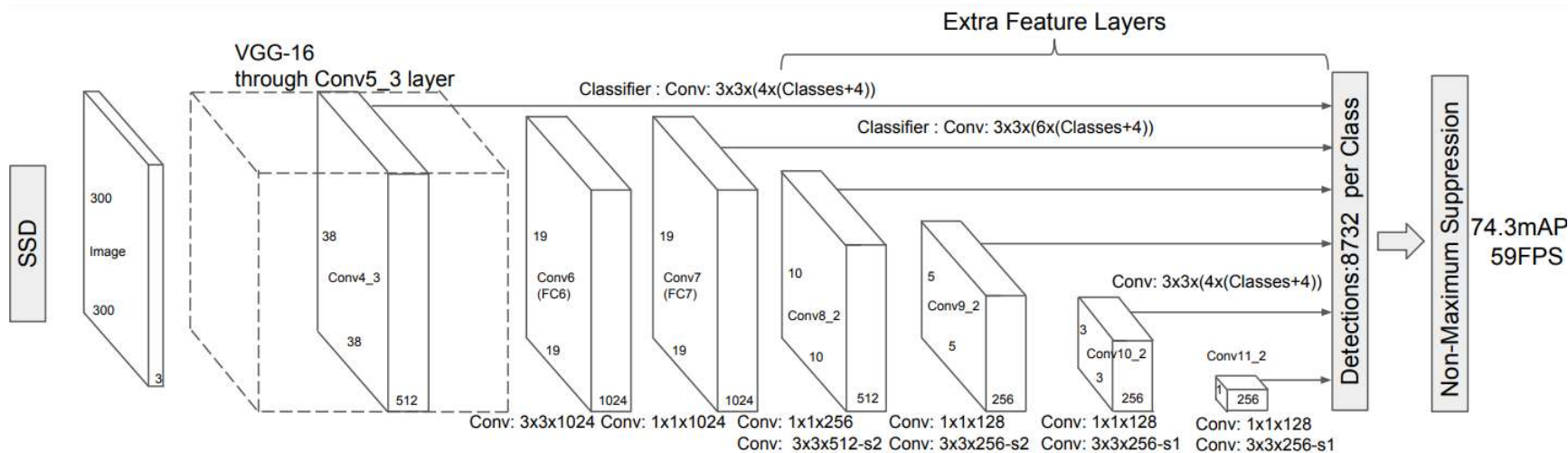
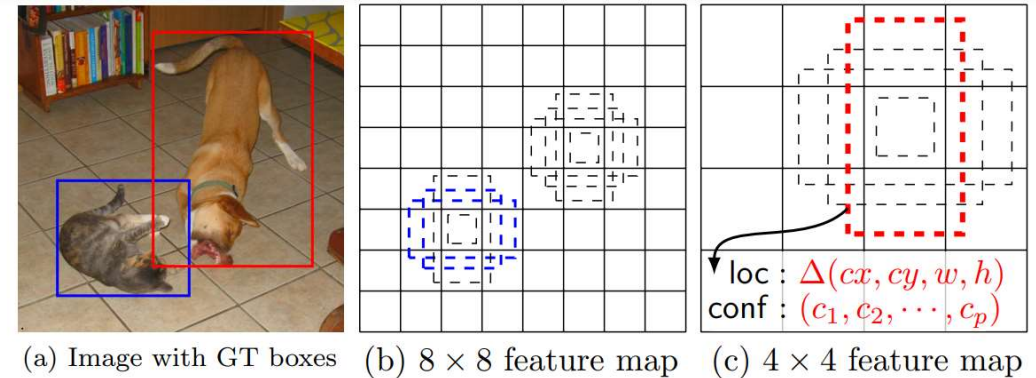
- バウンディングボックスを検出と分類を同時に予測する
- YOLOよりも計算が早く、Faster R-CNNと同等の精度を出した

- SSD以前モデルの課題点
- **SSDの推論**
- SSDの学習
- 実験結果

SSDの推論

■ 推論の流れ

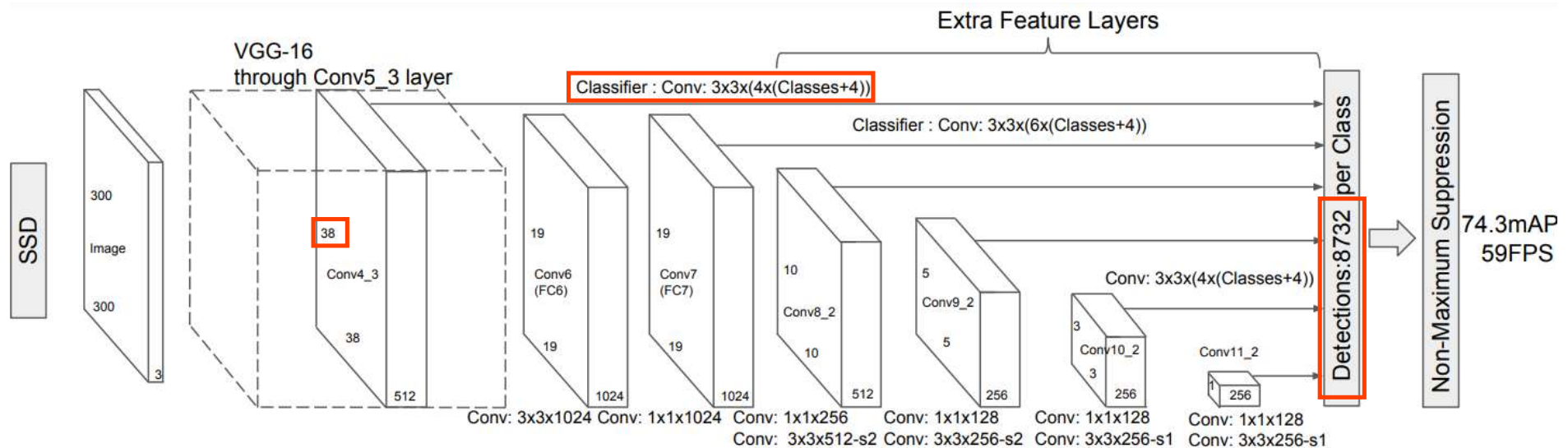
1. 異なる大きさに特徴マップに分割する
2. それぞれの特徴マップのピクセルごとに、アスペクト比で4~6種類のデフォルトボックスを生成する
3. デフォルトボックスに対して、画像の分類（クラス分類問題）と、大きさと位置（回帰問題）を推測する
4. 後処理



SSDの推論

■ 畳み込みフィルタ

- 3×3の畳み込みフィルタを利用している
- 画像を縦横38,19,10,5,3,1分割して特徴マップごと畳み込みをする
- 特徴マップの1ピクセルの畳み込みフィルタの出力は
(4~6種類のデフォルトボックス) × (画像分類(classes)+画像の位置(2)+画像の高さと幅(2))
- 生成されるデフォルトボックスの総数は (特徴マップのピクセル数) × (デフォルトボックスの種類)
 $38 \times 38 \times 4 + 19 \times 19 \times 6 + 10 \times 10 \times 6 + 5 \times 5 \times 6 + 1 \times 1 \times 4 = 8732$



- SSD以前モデルの課題点
- SSDの推論
- **SSDの学習**
- 実験結果

SSDの学習

■ 学習するデータ

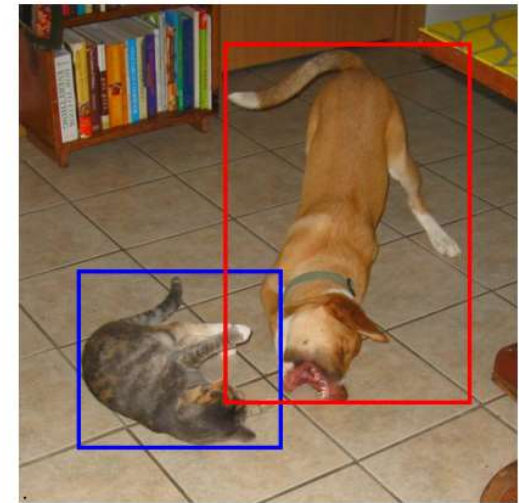
- 学習データはGTボックスとそのカテゴリを持つ
- 8732個の全てのデフォルトボックスを学習させるのではなく、一部を近しいGTボックスとマッチングさせて学習する

■ マッチング

- マッチングの評価指標としてJaccard係数を用いる

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- 各GTボックスを最大のJaccard係数を持つデフォルトボックスをマッチングさせて、閾値（0.5）よりも大きいJaccard係数を持つGTボックスにデフォルトボックスをマッチングさせる
- マッチングしたデフォルトボックスをポジティブ、残りをネガティブとして扱う



(a) Image with GT boxes

$$J(A, B) = \frac{\text{Intersection of two overlapping blue rectangles}}{\text{Union of two overlapping blue rectangles}}$$

■ 損失関数

- i 番目のデフォルトボックスが、クラス p の j 番目のGTボックスにマッチするかを $x_{ij}^p = \{0,1\}$ で表す
- N はポジティブなデフォルトボックスの数
- c は各クラスの信頼度、 l は予測ボックス、 g はGTボックス
- 損失関数は、ボックス推定の損失と、クラス分類の損失からなる

$$L(x, c, l, g) = \frac{1}{N} (L_{conf}(x, c) + \alpha L_{loc}(x, l, g))$$

- α は交差検証から1とする

SSDの学習

■ ボックス推定の損失

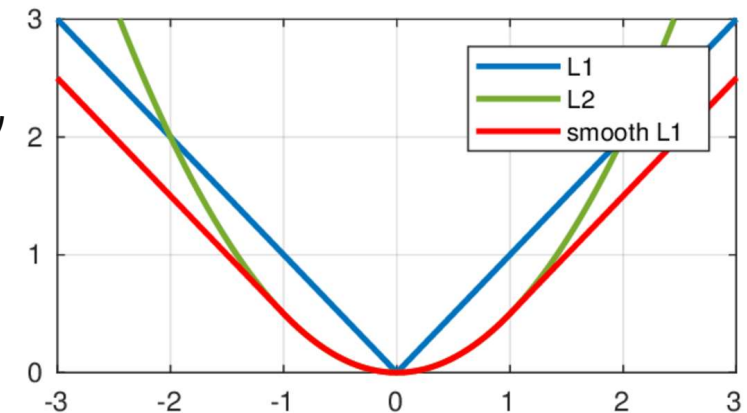
- (cx, cy) は予測ボックスの中心、 w は幅、 h は高さ
- ボックス推定の損失は、ポジティブな推定ボックスとGTボックスの差分のsmooth L1 loss

$$L_{loc}(x, l, g) = \sum_{i \in Pos} \sum_{m \in \{cx, cy, w, h\}} x_{ij}^k \text{smooth}_{L1}(l_i^m - \hat{g}_j^m)$$

- l_i^{cx} 、 l_i^{cy} はデフォルトボックスの中心から差分、 l_i^w 、 l_i^h はデフォルトボックスから何倍したかを対数で表す。 $(p$ 倍したら $\log p)$
- GTボックスは次のようにデフォルトボックス d_i^m で正規化する。

$$\hat{g}_j^{cx} = (g_j^{cx} - d_i^{cx})/d_i^w \quad \hat{g}_j^{cy} = (g_j^{cy} - d_i^{cy})/d_i^h$$

$$\hat{g}_j^w = \log \left(\frac{g_j^w}{d_i^w} \right) \quad \hat{g}_j^h = \log \left(\frac{g_j^h}{d_i^h} \right)$$



■ クラス分類の損失

- クラス分類の損失は、ポジティブな推定ボックスの分類に対する交差エントロピーと、ネガティブなボックスの分類に対する交差エントロピーからなる

$$L_{conf}(x, c) = - \sum_{i \in Pos}^N x_{ij}^p \log(\hat{c}_i^p) - \sum_{i \in Neg} \log(\hat{c}_i^0)$$

- ネガティブなボックスは背景クラス($p = 0$)に属することを正解とする
- $\hat{c}_i^p$ は各クラスの信頼度 c_i^p でソフトマックスを取った

$$\hat{c}_i^p = \frac{\exp(c_i^p)}{\sum_p \exp(c_i^p)}$$

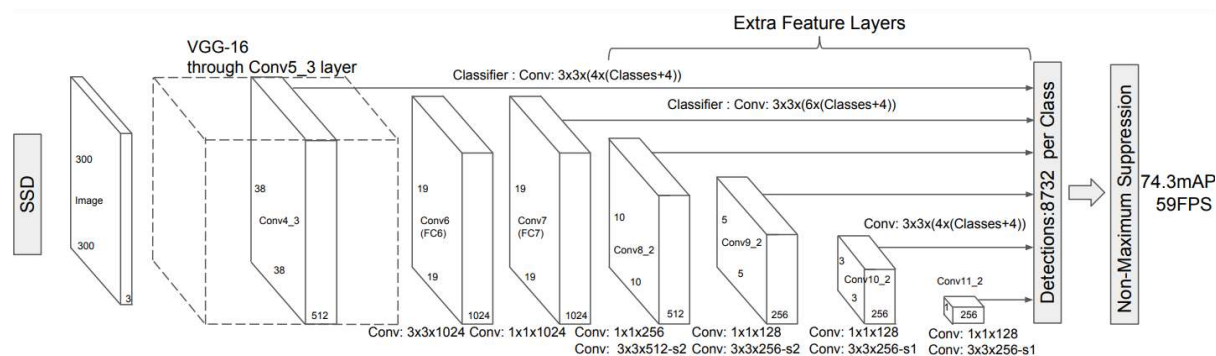
SSDの学習

■ デフォルトボックスのスケール

- 特徴マップに分割するスケールによって、デフォルトボックスのスケールを決定する。
- 分割する特徴マップの種類が m 個、特徴マップが小さい順に k 番目とすると、デフォルトボックスのスケールは

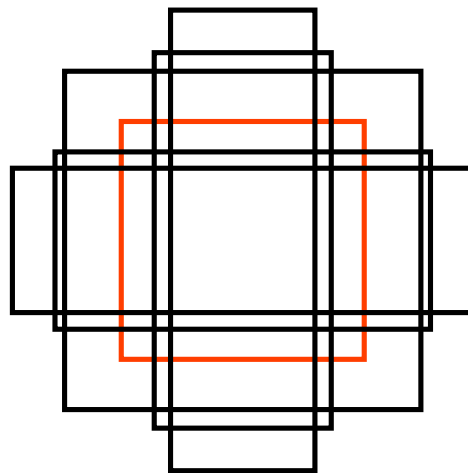
$$s_k = s_{\min} + \frac{s_{\max} - s_{\min}}{m - 1}(k - 1), \quad k \in [1, m]$$

- このモデルでは、 $s_{\min} = 0.2, s_{\max} = 0.9$ とした（1.0は画像全体の大きさ）



SSDの学習

- デフォルトボックスのアスペクト比
 - アスペクト比は $a_r \in \{1, 2, 3, \frac{1}{2}, \frac{1}{3}\}$ がある
 - $w_k^a = s_k \sqrt{a_r}, h_k^a = s_k / \sqrt{a_r}$ と幅、高さが表される
 - $a_r = 1$ のとき、スケールが $s'_k = \sqrt{s_k s_{k+1}}$ のデフォルトボックスも追加する
 - 計6種類のデフォルトボックスがある



赤のボックスが
 $a_r = 1$

SSDの学習

■ ハードネガティブマイニング

- ネガティブなボックスが多いので、ポジティブなボックスとネガティブなボックスの比が1:3になるように、ネガティブなボックスのクラス分類の損失 $-\log(\hat{c}_i^0)$ が大きい順に選ぶ。
- つまり、ネガティブなボックスのうち背景クラスの確率が低いもので学習する

■ データ補強

- そのまま
- 画像の切り取りとGTボックスのJaccard係数の最小値が0.1,0.3,0.5,0.7,0.9になるようにサンプリング
- ランダムに画像を切り取りサンプリング
- 画像の反転など

- SSD以前モデルの課題点
- SSDの推論
- SSDの学習
- **実験結果**

実験結果

■ 精度

Method	data	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
Fast [6]	07	66.9	74.5	78.3	69.2	53.2	36.6	77.3	78.2	82.0	40.7	72.7	67.9	79.6	79.2	73.0	69.0	30.1	65.4	70.2	75.8	65.8
Fast [6]	07+12	70.0	77.0	78.1	69.3	59.4	38.3	81.6	78.6	86.7	42.8	78.8	68.9	84.7	82.0	76.6	69.9	31.8	70.1	74.8	80.4	70.4
Faster [2]	07	69.9	70.0	80.6	70.1	57.3	49.9	78.2	80.4	82.0	52.2	75.3	67.2	80.3	79.8	75.0	76.3	39.1	68.3	67.3	81.1	67.6
Faster [2]	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
Faster [2]	07+12+COCO	78.8	84.3	82.0	77.7	68.9	65.7	88.1	88.4	88.9	63.6	86.3	70.8	85.9	87.6	80.1	82.3	53.6	80.4	75.8	86.6	78.9
SSD300	07	68.0	73.4	77.5	64.1	59.0	38.9	75.2	80.8	78.5	46.0	67.8	69.2	76.6	82.1	77.0	72.5	41.2	64.2	69.1	78.0	68.5
SSD300	07+12	74.3	75.5	80.2	72.3	66.3	47.6	83.0	84.2	86.1	54.7	78.3	73.9	84.5	85.3	82.6	76.2	48.6	73.9	76.0	83.4	74.0
SSD300	07+12+COCO	79.6	80.9	86.3	79.0	76.2	57.6	87.3	88.2	88.6	60.5	85.4	76.7	87.5	89.2	84.5	81.4	55.0	81.9	81.5	85.9	78.9
SSD512	07	71.6	75.1	81.4	69.8	60.8	46.3	82.6	84.7	84.1	48.5	75.0	67.4	82.3	83.9	79.4	76.6	44.9	69.9	69.1	78.1	71.8
SSD512	07+12	76.8	82.4	84.7	78.4	73.8	53.2	86.2	87.5	86.0	57.8	83.1	70.2	84.9	85.2	83.9	79.7	50.3	77.9	73.9	82.5	75.3
SSD512	07+12+COCO	81.6	86.6	88.3	82.4	76.0	66.3	88.6	88.9	89.1	65.1	88.4	73.6	86.5	88.9	85.3	84.6	59.1	85.0	80.4	87.4	81.2

Table 1: PASCAL VOC2007 test detection results. Both Fast and Faster R-CNN

Method	data	Avg. Precision, IoU:			Avg. Precision, Area:			Avg. Recall, #Dets:			Avg. Recall, Area:		
		0.5:0.95	0.5	0.75	S	M	L	1	10	100	S	M	L
Fast [6]	train	19.7	35.9	-	-	-	-	-	-	-	-	-	-
Fast [24]	train	20.5	39.9	19.4	4.1	20.0	35.8	21.3	29.5	30.1	7.3	32.1	52.0
Faster [2]	trainval	21.9	42.7	-	-	-	-	-	-	-	-	-	-
ION [24]	train	23.6	43.2	23.6	6.4	24.1	38.3	23.2	32.7	33.5	10.1	37.7	53.6
Faster [25]	trainval	24.2	45.3	23.5	7.7	26.4	37.1	23.8	34.0	34.6	12.0	38.5	54.4
SSD300	trainval35k	23.2	41.2	23.4	5.3	23.2	39.6	22.5	33.2	35.3	9.6	37.6	56.5
SSD512	trainval35k	26.8	46.5	27.8	9.0	28.9	41.9	24.8	37.5	39.8	14.0	43.5	59.0

Table 5: COCO test-dev2015 detection results.

実験結果

■ SSDの構成要素の貢献度

	SSD300				
more data augmentation?		✓	✓	✓	✓
include $\{\frac{1}{2}, 2\}$ box?	✓		✓	✓	✓
include $\{\frac{1}{3}, 3\}$ box?	✓			✓	✓
use atrous?	✓	✓	✓		✓
VOC2007 test mAP	65.5	71.6	73.7	74.2	74.3

- Data augmentationとは、データ補強
- Atrous Convolutionとは、隙間をあける畳み込み

実験結果

■ 計算速度と精度

Method	mAP	FPS	batch size	# Boxes	Input resolution
Faster R-CNN (VGG16)	73.2	7	1	~ 6000	~ 1000 × 600
Fast YOLO	52.7	155	1	98	448 × 448
YOLO (VGG16)	66.4	21	1	98	448 × 448
SSD300	74.3	46	1	8732	300 × 300
SSD512	76.8	19	1	24564	512 × 512
SSD300	74.3	59	8	8732	300 × 300
SSD512	76.8	22	8	24564	512 × 512

- FPSは1秒あたりに処理できる画像の枚数

- [1512.02325\(SSD\)](#)
- <https://qiita.com/ikeyasu/items/a95448254dff958a05b5>
- <https://calc-life.hatenablog.com/entry/2022/10/01/144518>